

Lecture 8: Planning and Models

UCL COMP0089 — Reinforcement Learning

Lecturer: Matteo Hessel (2021)

Lecture Notes

Contents

1	Recap and Motivation	2
1.1	Dynamic Programming vs. Model-Free RL	2
1.2	Why Consider Model-Based RL?	2
2	Learning a Model	2
2.1	What is a Model?	2
2.2	Model Learning as Supervised Learning	3
2.3	Types of Models	3
2.4	Specific Model Architectures	4
3	Planning for Credit Assignment	4
3.1	DP with a Learned Model	4
3.2	Sample-Based Planning	4
3.3	Limits of Planning with an Inaccurate Model	5
4	Integrating Learning and Planning: Dyna	5
5	Planning and Experience Replay	6
5.1	Parametric Model vs. Experience Replay	6
6	Planning for Action Selection	7
6.1	Forward Search	7
6.2	Simulation-Based Search	7
6.2.1	Monte-Carlo Simulation for Prediction	7
6.2.2	Monte-Carlo Simulation for Control	8
6.3	Monte-Carlo Tree Search (MCTS)	8
6.4	Value Function Approximation in Search Trees	9
7	Summary	9

1 Recap and Motivation

Previous lectures covered:

- (i) *Bandits*: trading off exploration and exploitation.
- (ii) *Dynamic Programming*: solving prediction and control given full knowledge of the environment.
- (iii) *Model-free prediction and control*: solving prediction and control from interaction with the environment.
- (iv) *Function approximation*: generalising learned value functions to large state spaces.

1.1 Dynamic Programming vs. Model-Free RL

Definition 1.1 (Dynamic Programming Approach). In the **Dynamic Programming** (DP) paradigm the agent is given a model (i.e. complete knowledge of $p(s', r \mid s, a)$) and solves the MDP without needing to interact with the world.

Definition 1.2 (Model-Free RL). In **Model-Free RL** the agent has no model of the environment. Instead, it learns value functions (and/or policies) directly from experience collected by interacting with the environment.

Definition 1.3 (Model-Based RL). In **Model-Based RL** the agent *learns* a model from experience and then *plans* value functions and/or policies using the learned model.

Remark 1.1 (Model-Free vs. Model-Based Cycle). Model-free RL operates in a *two-node* cycle: $\text{experience} \xrightarrow{\text{learning}} \text{value functions} \xrightarrow{\text{acting}} \text{experience}$. Model-based RL introduces a third node: $\text{experience} \xrightarrow{\text{learning}} \text{model} \xrightarrow{\text{planning}} \text{value functions} \xrightarrow{\text{acting}} \text{experience}$. A hybrid architecture (see Dyna, Algorithm 4.1) adds a direct learning arrow from experience to value functions alongside the planning arrow.

1.2 Why Consider Model-Based RL?

Remark 1.2 (Trade-offs of Model-Based RL). A clear disadvantage of model-based RL is the introduction of *two* sources of approximation error—one from learning the model $\hat{p}_\eta \approx p$ and one from computing the value function from $\hat{p}_\eta$ —whereas model-free RL incurs only one source of approximation error.

However, model-based RL offers several potential advantages:

- (a) Models can be learned efficiently via *supervised learning* methods.
- (b) One can reason about *model uncertainty* (e.g. via Bayesian methods), which can improve exploration.
- (c) Fewer interactions with the real environment are needed (*data efficiency*), which is especially important when real-world interaction is expensive, slow, or unsafe.

2 Learning a Model

2.1 What is a Model?

Definition 2.1 (Environment Model). A **model** $\mathcal{M}_\eta$ is an approximate representation of an MDP $\langle \mathcal{S}, \mathcal{A}, \hat{p} \rangle$, parametrised by weights η . We assume the state and action spaces coincide with those of the true MDP and that the dynamics are approximated by

$$R_{t+1}, S_{t+1} \sim \hat{p}_\eta(r, s' \mid S_t, A_t) \approx p(r, s' \mid S_t, A_t). \quad (1)$$

2.2 Model Learning as Supervised Learning

Remark 2.1 (Model Learning Objective). Given experience $\{S_1, A_1, R_2, S_2, \dots, S_T\}$, learning the model (Definition 2.1) reduces to a supervised learning problem over a dataset of state transitions:

$$(S_1, A_1) \rightarrow (R_2, S_2), \quad (S_2, A_2) \rightarrow (R_3, S_3), \quad \dots, \quad (S_{T-1}, A_{T-1}) \rightarrow (R_T, S_T).$$

One chooses a functional form $f_\eta(s, a) = (r, s')$, a loss function (e.g. mean-squared error), and finds parameters η that minimise the empirical loss.

2.3 Types of Models

Definition 2.2 (Expectation Model). An **expectation model** is a deterministic function $f_\eta(s, a) = (r, s')$ whose output approximates the *conditional expectation* of the next state and reward:

$$s' \approx \mathbb{E}[S_{t+1} \mid S_t = s, A_t = a].$$

Remark 2.2 (Disadvantage of Expectation Models). Expectation models can produce *invalid* states. For instance, if an action randomly transitions left or right past a wall, the expected next state may lie *inside* the wall.

Proposition 2.1 (Compatibility with Linear Values). *Let $\phi_t = \phi(S_t)$ be a feature representation and suppose the value function is linear: $v_\theta(\phi_t) = \theta^\top \phi_t$. Then, for an expectation model $f_\eta(\phi_t) = \mathbb{E}[\phi_{t+1} \mid S_t = s]$,*

$$\mathbb{E}[v_\theta(\phi_{t+1}) \mid S_t = s] = \mathbb{E}[\theta^\top \phi_{t+1} \mid S_t = s] = \theta^\top \mathbb{E}[\phi_{t+1} \mid S_t = s] = v_\theta(\mathbb{E}[\phi_{t+1} \mid S_t = s]). \quad (2)$$

If the model is also linear, $f_\eta(\phi_t) = P\phi_t$ for some matrix P , then one can unroll the expectation model multiple steps and still have $\mathbb{E}[v_\theta(\phi_{t+n}) \mid S_t = s] = v_\theta(\mathbb{E}[\phi_{t+n} \mid S_t = s])$.

Proof. Equation (2) follows directly from the linearity of both the expectation operator and v_θ . For the multi-step case, linearity of P gives $\mathbb{E}[\phi_{t+n} \mid S_t = s] = P^n \phi(s)$, and therefore $\mathbb{E}[v_\theta(\phi_{t+n}) \mid S_t = s] = \theta^\top P^n \phi(s) = v_\theta(P^n \phi(s))$. $\square$

Definition 2.3 (Stochastic (Generative) Model). A **stochastic model** (also called a *generative model*) samples next states and rewards via

$$\hat{R}_{t+1}, \hat{S}_{t+1} = \hat{p}(S_t, A_t, \omega), \quad (3)$$

where ω is a noise term. Stochastic models can be chained even when the dynamics are non-linear, at the cost of introducing sampling noise.

Definition 2.4 (Full (Distribution) Model). A **full model** represents the complete transition distribution $\hat{p}(s' \mid s, a)$ and reward function $\hat{r}(s, a, s')$. Computing the expected value under a full model requires summation over all successor states:

$$\mathbb{E}[v(S_{t+1}) \mid S_t = s] = \sum_a \pi(a \mid s) \sum_{s'} \hat{p}(s' \mid s, a) (\hat{r}(s, a, s') + \gamma v(s')). \quad (4)$$

Multi-step unrolling causes exponential branching, since each level of the sum introduces a new summation over states and actions.

2.4 Specific Model Architectures

Definition 2.5 (Table Lookup Model). A **table lookup model** constructs an explicit MDP by counting visits to each state–action pair. Let $N(s, a)$ denote the number of times (s, a) has been visited. The empirical transition and reward models are:

$$\hat{p}_t(s' | s, a) = \frac{1}{N(s, a)} \sum_{k=0}^{t-1} \mathbb{I}(S_k = s, A_k = a, S_{k+1} = s'), \quad (5)$$

$$\mathbb{E}_{\hat{p}_t}[R_{t+1} | S_t = s, A_t = a] = \frac{1}{N(s, a)} \sum_{k=0}^{t-1} \mathbb{I}(S_k = s, A_k = a) R_{k+1}. \quad (6)$$

Example 2.1 (AB Example). Consider two states A, B with no discounting ($\gamma = 1$) and 8 episodes of experience:

$$A, 0, B, 0 \mid B, 1 \mid B, 1 \mid B, 1 \mid B, 1 \mid B, 1 \mid B, 1 \mid B, 0.$$

Applying the table lookup model (Definition 2.5):

- From A : deterministically transitions to B with reward 0 (observed once).
- From B : transitions to a terminal state with reward 1 with probability $\frac{6}{8} = 75\%$, and with reward 0 with probability $\frac{2}{8} = 25\%$.

Definition 2.6 (Linear Expectation Model). Given a feature representation $\phi(s)$, a **linear expectation model** parametrises transitions and rewards separately for each action a :

$$\hat{s}'(s, a) = T_a \phi(s), \quad (7)$$

$$\hat{r}(s, a) = w_a^\top \phi(s), \quad (8)$$

where T_a is a square matrix and w_a is a weight vector. On each observed transition (s, a, r, s') , the parameters are updated by gradient descent to minimise

$$L(s, a, r, s') = \|\phi(s') - T_a \phi(s)\|^2 + (r - w_a^\top \phi(s))^2. \quad (9)$$

3 Planning for Credit Assignment

Definition 3.1 (Planning). **Planning** is the process of investing computation to improve value functions and/or policies *without* interacting with the environment. Dynamic programming (which requires a perfect model) is a canonical example. The planning algorithms discussed here use *learned* models (Definition 2.1) rather than requiring privileged access to the true dynamics p .

3.1 DP with a Learned Model

Remark 3.1 (DP on the Approximate MDP). Once a model $\hat{p}_\eta$ has been learned from experience (Section 2), one can solve the approximate MDP $\langle \mathcal{S}, \mathcal{A}, \hat{p}_\eta \rangle$ using any standard DP algorithm (e.g. value iteration, policy iteration).

3.2 Sample-Based Planning

Definition 3.2 (Sample-Based Planning). In **sample-based planning**, the learned model is used solely to *generate samples*:

$$S', R \sim \hat{p}_\eta(\cdot | s, a).$$

Model-free RL algorithms (e.g. Monte-Carlo control, Sarsa, Q-learning) are then applied to these simulated transitions, exactly as if they were real experience.

Example 3.1 (Sample-Based Planning on the AB Example). Returning to the AB example (Example 2.1): a table-lookup model is constructed from the real experience, then model-free RL (e.g. Monte-Carlo learning) is applied to sampled experience generated from the model. This yields value estimates such as $V(A) = 1$ and $V(B) = 0.75$.

3.3 Limits of Planning with an Inaccurate Model

Remark 3.2 (Model Accuracy Bound). Given an imperfect model $\hat{p}_\eta \neq p$, the planning process computes a policy that is optimal only with respect to the approximate MDP $\langle \mathcal{S}, \mathcal{A}, \hat{p}_\eta \rangle$. The performance of model-based RL is therefore bounded by the quality of the learned model.

Remark 3.3 (Dealing with Model Inaccuracy). Three approaches to handle inevitable model inaccuracies:

- (a) Fall back to model-free RL when the model is known to be wrong.
- (b) Reason about model uncertainty over η (e.g. Bayesian methods).
- (c) Combine model-based and model-free methods in a single algorithm (see Dyna, Algorithm 4.1).

4 Integrating Learning and Planning: Dyna

Definition 4.1 (Real vs. Simulated Experience). We distinguish two sources of experience:

- **Real experience**: sampled from the true environment, $r, s' \sim p(\cdot | s, a)$.
- **Simulated experience**: sampled from the learned model, $r, s' \sim \hat{p}_\eta(\cdot | s, a)$.

Definition 4.2 (Dyna Architecture). The **Dyna** architecture (Sutton, 1991) integrates learning and planning by:

- (i) Learning a model from real experience.
- (ii) Learning *and* planning value functions (and/or policies) from *both* real and simulated experience.

Conceptually, updates from real experience (“learning”) and from simulated experience (“planning”) are treated identically. This contrasts with pure model-free RL (Definition 1.2), which uses only real experience, and pure model-based sample-based planning (Definition 3.2), which uses only simulated experience.

Algorithm 4.1 (Dyna-Q). The **Dyna-Q** algorithm (assuming a deterministic environment):

- (a) Initialise $Q(s, a)$ and $\text{Model}(s, a)$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$.
- (b) **Do forever**:
 - (1) $s \leftarrow$ current (non-terminal) state.
 - (2) $a \leftarrow \varepsilon$ -greedy(s, Q).
 - (3) Execute action a ; observe resultant state s' and reward r .
 - (4) **Direct RL update**:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (10)$$

- (5) **Model update**: $\text{Model}(s, a) \leftarrow (s', r)$.

- (6) **Planning:** Repeat N times:
- (i) $\tilde{s} \leftarrow$ random previously observed state.
 - (ii) $\tilde{a} \leftarrow$ random action previously taken in $\tilde{s}$.
 - (iii) $\tilde{s}', \tilde{r} \leftarrow \text{Model}(\tilde{s}, \tilde{a})$.
 - (iv) $Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha[\tilde{r} + \gamma \max_{a'} Q(\tilde{s}', a') - Q(\tilde{s}, \tilde{a})]$.

Note that step (4) is a standard Q-learning update (direct RL), while step (6) performs N additional Q-learning updates using *simulated* transitions from the model.

Remark 4.1 (Advantages of Dyna). By investing additional computation (step (6) in Algorithm 4.1), the agent learns more efficiently from fewer real-world interactions. This is especially important when collecting real data is expensive, slow (e.g. robotics), or unsafe (e.g. autonomous driving).

Example 4.1 (Dyna-Q on a Simple Maze). In a gridworld maze experiment, Dyna-Q with $N = 50$ planning steps converges to the optimal path length (≈ 14 steps per episode) within roughly 5 episodes. With $N = 5$ planning steps, convergence takes about 20 episodes. With $N = 0$ (pure Q-learning, no planning), convergence requires over 25 episodes and exhibits much higher variance.

Example 4.2 (Dyna-Q with an Inaccurate Model—Harder Environment). When the environment changes mid-training to become *harder* (e.g. new walls blocking the previously optimal path), Dyna-Q+ (which adds an exploration bonus for actions not taken recently) adapts faster than standard Dyna-Q, which in turn outperforms Dyna-AC.

Example 4.3 (Dyna-Q with an Inaccurate Model—Easier Environment). When the environment changes to become *easier* (e.g. a shortcut opens up), Dyna-Q+ may initially lag behind Dyna-Q, since its exploration bonus causes it to explore suboptimal paths. However, it eventually discovers and exploits the shortcut.

5 Planning and Experience Replay

Remark 5.1 (Beyond the Model-Based / Model-Free Dichotomy). The sharp distinction between model-based and model-free methods is now less useful in practice:

- (1) Modern agents often store transitions in an *experience replay buffer*.
- (2) Model-free RL is applied to experience sampled from this buffer.
- (3) This is equivalent to Dyna (Definition 4.2) with the replay buffer serving as a *non-parametric model*: planning is performed by sampling complete transitions (s, a, r, s') rather than querying a parametric model for (r, s') given (s, a) .

The agent can invest more compute by making many updates on past data for every new step in the environment, mirroring the planning loop in Dyna.

5.1 Parametric Model vs. Experience Replay

Remark 5.2 (Tabular Equivalence). In the tabular setting, if the model is perfect (i.e. converged to the true transition counts), it produces the same output as a non-parametric replay system for every (s, a) pair. In practice the model is imperfect, so there will be differences.

Remark 5.3 (Flexibility of Parametric Models). A parametric model is more flexible than an experience replay buffer in that it supports:

- (a) **Planning for action selection:** querying the model for actions one *could* take in the future.
- (b) **Counterfactual planning:** querying the model for actions one *could have* taken in the past but did not.
- (c) **Backwards planning:** modelling inverse dynamics and assigning credit to states that *could have* led to a certain outcome.
- (d) **Jumpy planning:** planning at different time scales for long-term credit assignment.

Remark 5.4 (Computation and Memory Trade-offs). *Computation:* querying a replay buffer is very cheap, whereas generating a sample from a large neural-network-based generative model can be expensive.

Memory: the memory requirements of a replay buffer scale linearly with its capacity, whereas a parametric model can achieve good accuracy with a fixed and comparably small memory footprint.

6 Planning for Action Selection

Remark 6.1 (Local vs. Global Planning). So far we considered planning to improve a *global* value function. An alternative is to plan for the *near future* to select the next action. Since the distribution of states reachable from the current state can differ substantially from the distribution of states reachable from a starting state, the agent may be able to construct a more accurate *local* value function. Furthermore, inaccuracies in the model may lead to interesting exploration rather than harmful updates.

6.1 Forward Search

Definition 6.1 (Forward Search). **Forward search** algorithms select the best action by *lookahead*. They build a search tree rooted at the current state s_t using a model of the MDP to simulate future trajectories. Only the sub-MDP starting from s_t needs to be solved, rather than the entire MDP.

6.2 Simulation-Based Search

Definition 6.2 (Simulation-Based Search). **Simulation-based search** is a sample-based variant of forward search (Definition 6.1). It simulates episodes of experience from the current state using the model and applies model-free RL to the simulated episodes.

6.2.1 Monte-Carlo Simulation for Prediction

Proposition 6.1 (MC Simulation for State Evaluation). *Given a model $\mathcal{M}_\eta$ (Definition 2.1) and a simulation policy π , simulate K episodes from the current state S_t :*

$$\{S_t^k = S_t, A_t^k, R_{t+1}^k, S_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \hat{p}_\eta, \pi.$$

The state value is estimated by the mean return:

$$v(S_t) = \frac{1}{K} \sum_{k=1}^K G_t^k \approx v_\pi(S_t). \quad (11)$$

6.2.2 Monte-Carlo Simulation for Control

Proposition 6.2 (MC Simulation for Action Selection). *For each action $a \in \mathcal{A}$, simulate K episodes from the current state s with the first action fixed to a :*

$$\{S_t^k = s, A_t^k = a, R_{t+1}^k, S_{t+1}^k, A_{t+1}^k, \dots, S_T^k\}_{k=1}^K \sim \mathcal{M}_\eta, \pi.$$

The action value is estimated by the mean return:

$$q(s, a) = \frac{1}{K} \sum_{k=1}^K G_t^k \approx q_\pi(s, a). \quad (12)$$

The real action is selected greedily:

$$A_t = \arg \max_{a \in \mathcal{A}} q(S_t, a). \quad (13)$$

6.3 Monte-Carlo Tree Search (MCTS)

Definition 6.3 (Monte-Carlo Tree Search). **Monte-Carlo Tree Search** (MCTS) incrementally builds a search tree (Definition 6.1) containing visited state-action pairs together with estimated action values $q(s, a)$. Each simulation consists of four phases:

- (i) **Select**: Starting from the root, follow the tree policy (based on $q(s, a)$) until a leaf node is reached.
- (ii) **Expand**: Add one new node to the search tree.
- (iii) **Rollout**: From the new node, simulate until episode termination using a fixed *rollout (default) policy* (often uniform random).
- (iv) **Backup**: Update the action-value estimates for all state-action pairs visited in the tree:

$$q(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{u=t}^T \mathbb{I}(S_u^k = s, A_u^k = a) G_u^k \approx q_\pi(s, a). \quad (14)$$

When the computational budget is exhausted, the action at the root with the highest $q(s_t, a)$ (or highest visit count) is selected as the real action.

Remark 6.2 (Two Simulation Policies in MCTS). MCTS effectively maintains two simulation policies:

- (a) A **tree policy** that is refined during search (e.g. UCB1-based selection within the tree).
- (b) A **rollout (default) policy** that is held fixed; often this is simply uniform random action selection.

Remark 6.3 (Advantages of MCTS). Key advantages of Monte-Carlo Tree Search (Definition 6.3):

- (a) Highly selective best-first search.
- (b) Evaluates states dynamically (unlike, e.g., DP which sweeps over all states).
- (c) Uses sampling to break the curse of dimensionality.
- (d) Works for “black-box” models—only requires the ability to generate samples.
- (e) Computationally efficient, anytime (can be stopped at any point), and parallelisable.

6.4 Value Function Approximation in Search Trees

Remark 6.4 (Search Tree as Table Lookup). The search tree in MCTS is fundamentally a table-lookup approach (Definition 2.5), based on a partial instantiation of the value table. While table-lookup is naive for model-free RL (it cannot store values for all states and does not generalise between similar states), it is less naive for simulation-based search because the tree stores values only for *easily reachable* states from the current position.

However, in huge search spaces, the tree still does not generalise between similar states, and incorporating value function approximation into the search becomes beneficial.

7 Summary

This lecture introduced model-based reinforcement learning:

- (i) A *model* (Definition 2.1) approximates the environment dynamics and can be learned via supervised learning.
- (ii) Models come in several forms—expectation models (Definition 2.2), stochastic models (Definition 2.3), full distribution models (Definition 2.4), table-lookup models (Definition 2.5), and linear expectation models (Definition 2.6).
- (iii) *Planning* (Definition 3.1) uses the learned model to improve values/policies without further environment interaction.
- (iv) The *Dyna* architecture (Definition 4.2, Algorithm 4.1) integrates learning from real experience with planning from simulated experience.
- (v) Experience replay can be viewed as a non-parametric variant of Dyna (Remark 5.1).
- (vi) *Forward search* (Definition 6.1) and *Monte-Carlo Tree Search* (Definition 6.3) perform local planning rooted at the current state for action selection.